Clase 11 (Unidad 5) — GraphQL sobre el mismo dominio

Duración total: 4 horas | Proyecto: endpoint /graphql con Graphene-Django

Cómo leer esta guía

Cada parte combina dos tipos de contenido:

- **Conceptos** (qué es y por qué): la base teórica de la Unidad 5, con tablas y diagramas. Se pueden leer antes de la clase.
- **Práctica** (cómo): los pasos con código, que aplican esos conceptos a TaskFlow.

Al final hay **preguntas de repaso** para autoevaluarse.

Parte 0 — Antes de empezar (5 min)

Parados en `taskflow/`, entorno virtual activo. Dejen Redis corriendo (Clase 10): hoy no lo usamos directamente, pero la mutation que vamos a escribir reutiliza las notificaciones en tiempo real.

Parte 1 — GraphQL vs. REST (50 min)

1.1 El problema que ataca GraphQL

En REST, cada recurso responde con una forma fija de datos, definida por el serializer del lado del servidor. Si el frontend solo necesita el `title` de cada tarea, igual recibe todos los campos de `TaskSerializer` (**over-fetching**, traer de más). Si necesita datos de tareas **y** de sus proyectos asociados, muchas veces tiene que hacer **dos peticiones** separadas (**under-fetching**, traer de menos en una sola ida): `/api/tasks/` y después `/api/projects/<id>/`.

GraphQL invierte el control: el **cliente** describe, en el cuerpo de una única petición, exactamente qué campos y qué relaciones necesita, y el servidor responde exactamente eso, ni más ni menos. Una consulta GraphQL se parece a la forma del JSON que se quiere recibir:

```
query {
  allProjects {
    name
    tasks { title }
  }
}
```

Concepto: el sistema de tipos. Una API GraphQL se define con un **esquema** escrito en un lenguaje propio (SDL, *Schema Definition Language*). Este es el esquema real que vamos a construir hoy, tal como lo genera Graphene (resumido):

```
type Query {
  allProjects: [ProjectType]
  task(id: ID!): TaskType
  tasksByStatus(status: String!): [TaskType]
}

type ProjectType {
  id: ID!
  name: String!
  description: String!
  tasks: [TaskType!]!
}

type TaskType {
  id: ID!
  project: ProjectType!
  title: String!
  priority: String!
  status: String!
  dueDate: Date
```

```
  tags: [TagType!]!
}

type Mutation {
  createTask(priority: String = "media", projectId: ID!, title: String!): CreateTask
}
```

Cómo se lee:

- **Tipos escalares:** los valores simples. Los estándar son `Int`, `Float`, `String`, `Boolean` e `ID`; se pueden definir otros, como `Date`.
- **Tipos objeto:** conjuntos de campos (`ProjectType`, `TaskType`). Un campo puede ser de otro tipo objeto, y así se forman las relaciones.
- `!` significa "no puede ser nulo". `[ ]` significa lista: `[TaskType!]!` es una lista que siempre existe y cuyos elementos nunca son nulos.
- **Argumentos:** `task(id: ID!)` recibe un `id` obligatorio; `priority: String = "media"` tiene un valor por defecto.
- `Query` y `Mutation` son los puntos de entrada: todo lo que se puede leer y todo lo que se puede modificar. Existe un tercero, `Subscription`, para recibir datos en tiempo real (por WebSocket, igual que en la Unidad 4).

Concepto: la forma de una respuesta. Toda respuesta GraphQL tiene la misma forma: `{"data": ..., "errors": [...]}`. Puede traer las dos cosas a la vez: si una parte de la consulta falla, el resto igual se devuelve (**resultado parcial**). Por eso, en general, GraphQL responde `200` aunque haya errores, y el cliente tiene que mirar la lista `errors`.

Otros elementos del lenguaje que conviene conocer:

- **Variables:** en vez de escribir los valores dentro de la consulta, se mandan aparte, como parámetros (`query ($id: ID!) { task(id: $id) { title } }`). Es más seguro y permite reutilizar la consulta.
- **Alias:** permiten pedir el mismo campo dos veces con distintos argumentos (`pendientes: tasksByStatus(status: "pendiente") { title }`).
- **Fragmentos:** grupos de campos con nombre, para no repetirlos en varias consultas.

1.2 ¿Reemplaza a REST?

No necesariamente. Son dos estilos con fortalezas distintas:

- **REST** es simple, cacheable por HTTP de forma nativa (cada URL es un recurso identificable) y ampliamente entendido en la industria. Los códigos de estado HTTP (Clase 4) comunican el resultado.
- **GraphQL** brilla cuando el frontend necesita mucha flexibilidad sobre qué pedir (por ejemplo, distintas pantallas de una app móvil, cada una con necesidades de datos distintas), sin tener que crear un endpoint REST a medida para cada pantalla. Todo pasa por **una sola URL** (`/graphql/`), casi siempre con `POST`, y los errores viajan dentro del JSON de respuesta (en una lista `errors`), no como códigos HTTP.

Concepto: comparación detallada.

Aspecto	REST	GraphQL
Puntos de entrada	Muchas URLs, una por recurso	Una sola URL (`/graphql/`)
Quién decide qué datos vuelven	El servidor (el serializer)	El cliente (la consulta)
Datos relacionados	Varias peticiones o anidado fijo	Una petición, anidado a pedido
Errores	Códigos de estado HTTP	Lista `errors` en el cuerpo, con `200`
Caché	Nativa de HTTP, por URL	Más difícil: todo es un POST a la misma URL
Versionado	Habitual (`/api/v1/`, `/api/v2/`)	Se evita: se agregan campos y se marcan los viejos como obsoletos (`@deprecated`)

Documentación	Hace falta generarla (OpenAPI)	El esquema es la documentación (introspección)
Riesgos propios	Endpoints de más o de menos	Consultas demasiado profundas o costosas (Parte 3.3)

En esta clase los vamos a hacer **convivir** sobre exactamente los mismos modelos (`Project`, `Task`, `Tag`) y, muy importante, sobre **las mismas reglas de negocio y de permisos** que ya construimos. Una segunda puerta de entrada a los datos que se saltee los permisos de la Clase 7 sería un agujero de seguridad.

Parte 2 — Schema, queries y mutations con Graphene-Django (95 min)

2.1 Instalación

Parados en `taskflow/`, entorno virtual activo:

```
pip install graphene-django
pip freeze > requirements.txt
```

Al final de `taskflow/config/settings.py`:

```
# --- GraphQL (Clase 11) ---
INSTALLED_APPS += ["graphene_django"]

GRAPHENE = {
    "SCHEMA": "config.schema.schema",
}
```

`GRAPHENE["SCHEMA"]` le dice a Graphene-Django dónde va a estar el esquema final: la variable `schema` dentro del módulo `config/schema.py`, un archivo que todavía tenemos que crear (Parte 2.4).

2.2 Definir los tipos (Types)

Un `DjangoObjectType` le indica a Graphene cómo exponer un modelo de Django como un "tipo" de GraphQL. Es conceptualmente similar a lo que hace un `ModelSerializer` de DRF, pero para el mundo GraphQL.

Creen el archivo `taskflow/core/schema.py`:

```
import graphene
from graphene_django import DjangoObjectType
from graphql import GraphQLError

from .models import Project, Task, Tag
from .serializers import TaskSerializer
from .services import MembershipService, NotificationService


class TagType(DjangoObjectType):
    class Meta:
        model = Tag
        fields = ("id", "name")


class TaskType(DjangoObjectType):
    class Meta:
        model = Task
        fields = ("id", "title", "description", "priority", "status", "due_date", "tags", "project")
        convert_choices_to_enum = False  # "pendiente" en vez de "PENDIENTE", igual que en REST


class ProjectType(DjangoObjectType):
    class Meta:
        model = Project
```

```
        fields = ("id", "name", "description", "tasks")
```

- El patrón `class Meta: model = ...; fields = (...)` es análogo al `class Meta` de un `ModelSerializer` de DRF (Clase 3): qué modelo usar como base y qué campos exponer explícitamente. Otra vez, a propósito, evitamos exponer "todos los campos" sin pensarlo (por ejemplo, `owner` no está).
- `"tags"`, `"project"` y `"tasks"` son **relaciones**: Graphene las resuelve solo, usando el tipo correspondiente (`TagType`, `ProjectType`, `TaskType`). Por eso el cliente puede pedir `tasks { title }` dentro de un proyecto.
- `convert_choices_to_enum = False`: por defecto, Graphene convierte los campos con `choices` (como `status` y `priority`) en un *enum* de GraphQL, y los muestra en mayúsculas (`PENDIENTE`). Con esta opción los deja como texto, con los mismos valores que devuelve la API REST.
- Los imports de `GraphQLError`, `TaskSerializer` y los servicios los usamos en las partes siguientes.

2.3 Queries: leer datos

En el mismo archivo `taskflow/core/schema.py`, agreguen:

```
def tasks_visible_for(user):
    """Mismas reglas que TaskViewSet.get_queryset (Clase 7): solo tareas de mis proyectos."""
    if not user.is_authenticated:
        return Task.objects.none()
    return Task.objects.filter(project__memberships__user=user).distinct()


class Query(graphene.ObjectType):
    all_projects = graphene.List(ProjectType)
    task = graphene.Field(TaskType, id=graphene.ID(required=True))
    tasks_by_status = graphene.List(TaskType, status=graphene.String(required=True))

    def resolve_all_projects(root, info):
        user = info.context.user
        if not user.is_authenticated:
            return Project.objects.none()
        return MembershipService.projects_for(user)

    def resolve_task(root, info, id):
        return tasks_visible_for(info.context.user).filter(pk=id).first()

    def resolve_tasks_by_status(root, info, status):
        return tasks_visible_for(info.context.user).filter(status=status)
```

- `class Query(graphene.ObjectType)` define, campo por campo, qué se puede **consultar**. `graphene.List(ProjectType)` devuelve una lista; `graphene.Field(TaskType, id=...)` devuelve un solo elemento y recibe un argumento obligatorio `id`.
- Cada campo necesita un método `resolve_<nombre_del_campo>`: el **resolver**, la función que sabe obtener ese dato, de forma análoga a como una vista de DRF sabe resolver un `GET`.
- En GraphQL los nombres se escriben en *camelCase*: Graphene convierte automáticamente `all_projects` en `allProjects` y `tasks_by_status` en `tasksByStatus`.
- `info.context` es el equivalente, en GraphQL, al `request` de una vista DRF: de ahí sacamos `info.context.user`, el usuario autenticado (Parte 3.1 explica cómo llega el JWT hasta acá).
- `tasks_visible_for` aplica **la misma regla de membresía** que `TaskViewSet.get_queryset` de la Clase 7, y `resolve_all_projects` reutiliza `MembershipService.projects_for`. Así, `task(id: 5)` de un proyecto ajeno devuelve `null`, igual que el `404` de la API REST.
- `.first()` devuelve el primer resultado o `None` si no hay ninguno (en vez de lanzar una excepción como `.get()`).
- `Project.objects.none()` devuelve un queryset vacío explícito: la forma correcta en Django de decir "ninguno" sin romper el tipo esperado.

Concepto: resolvers y contexto de ejecución. El programa pide entender "resolvers y contexto de ejecución". GraphQL ejecuta una consulta como un **árbol**: resuelve primero los campos de arriba y, con cada resultado, los campos de adentro.

```
{ allProjects { name tasks { title } } }

resolve_all_projects(root=None, info)        → [proyecto 1, proyecto 2]
   ├─ name  de proyecto 1   (resolver por defecto: lee proyecto1.name)
   ├─ tasks de proyecto 1   (resolver por defecto: proyecto1.tasks.all())
   │     └─ title de cada tarea
   └─ ... lo mismo para proyecto 2
```

- Cada resolver recibe el **objeto padre** (`root`): para `allProjects` no hay padre (`None`); para `tasks`, el padre es el proyecto.
- Solo escribimos resolvers para los puntos de entrada. Los campos de un `DjangoObjectType` tienen **resolvers por defecto**, que leen el atributo del modelo con el mismo nombre.
- **`info.context`** es el **contexto de ejecución**: lo que comparten todos los resolvers de una misma consulta. En Graphene-Django es el `request` de Django, con el usuario autenticado. Es el lugar natural para todo lo que depende de "quién pregunta".

Concepto: el problema N+1 en GraphQL. Resolver campo por campo tiene un costo escondido. Para la consulta de arriba, con 6 proyectos, medimos en TaskFlow **7 consultas SQL**: 1 para los proyectos y 1 más **por cada proyecto** para sus tareas. Con 100 proyectos serían 101. Es el mismo problema N+1 de la Clase 3, pero en GraphQL es más fácil caer en él, porque el cliente decide qué anidar. Dos soluciones:

- **Cargar las relaciones por adelantado** en el resolver de arriba: `.prefetch_related("tasks")`. En la misma medición, bajó a **2 consultas**, sin importar la cantidad de proyectos.
- **DataLoader:** un patrón que junta todos los pedidos de un mismo nivel y los resuelve en una sola consulta. Es la solución estándar en las librerías de GraphQL de todos los lenguajes.

2.4 Mutations: escribir datos

Siguiendo en `taskflow/core/schema.py`:

```
class CreateTask(graphene.Mutation):
    class Arguments:
        project_id = graphene.ID(required=True)
        title = graphene.String(required=True)
        priority = graphene.String(default_value="media")

    task = graphene.Field(TaskType)
    ok = graphene.Boolean()

    def mutate(root, info, project_id, title, priority):
        user = info.context.user
        if not user.is_authenticated:
            raise GraphQLError("Autenticación requerida.")

        # Reutilizamos las validaciones del serializer de DRF (Clases 4 y 8)
        serializer = TaskSerializer(data={"project": project_id, "title": title, "priority": priority})
        if not serializer.is_valid():
            errores = {campo: [str(e) for e in lista] for campo, lista in serializer.errors.items()}
            raise GraphQLError(f"Datos inválidos: {errores}")

        # Misma regla de permisos que en REST (Clase 7)
        if not MembershipService.is_admin(user, serializer.validated_data["project"]):
            raise GraphQLError("Solo un admin del proyecto puede crear tareas en él.")

        task = serializer.save(owner=user)
        NotificationService.notify_task_created(task)  # Clase 10
        return CreateTask(task=task, ok=True)


class Mutation(graphene.ObjectType):
    create_task = CreateTask.Field()
```

```
schema = graphene.Schema(query=Query, mutation=Mutation)
```

Una **mutation** en GraphQL es el equivalente conceptual a un POST/PUT/DELETE en REST: una operación que **modifica** datos, a diferencia de una **query**, que solo lee.

- `class Arguments` declara qué parámetros recibe la mutation desde el cliente. `default_value="media"` hace que `priority` sea opcional.
- `task` y `ok` son los campos que la mutation **devuelve**; el cliente elige cuáles pedir.
- `mutate` ejecuta la acción real y **debe devolver una instancia de la propia clase** (`CreateTask(task=task, ok=True)`).
- En vez de crear la tarea con `Task.objects.create(...)` directamente, **pasamos los datos por `TaskSerializer`**. Así se aplican exactamente las mismas validaciones que en la API REST: que `priority` sea un valor permitido ("urgente" es rechazado), que el proyecto exista, la regla de `en_progreso` de la tarea de la Clase 4, la limpieza de HTML de la Clase 8. Si escribiéramos `Task.objects.create`, GraphQL sería una "puerta de atrás" que se saltea todas esas reglas.
- `serializer.errors` trae los mensajes de error como objetos de DRF; la comprensión de diccionario los convierte en texto simple para que el mensaje sea legible.
- `GraphQLError` es la forma de devolver un error en GraphQL: la respuesta llega con un `200` HTTP, pero con una lista `errors` en el JSON.
- La verificación de admin y la notificación son las mismas de `TaskViewSet.perform_create` (Clases 7 y 10), reutilizando los mismos servicios.
- `graphene.Schema(query=Query, mutation=Mutation)` combina lectura y escritura en un único esquema completo.

Concepto: cómo diseñar una mutation. Algunas convenciones muy difundidas:

- **Nombres con verbo:** `createTask`, `updateTaskStatus`, `archiveProject`. A diferencia de REST, en GraphQL cada operación de escritura es explícita (se parece más a las `@action` de la Clase 5 que al CRUD genérico).
- **Devolver el objeto modificado:** así el cliente actualiza su pantalla sin hacer otra consulta. Por eso `CreateTask` devuelve `task`.
- **Errores:** hay dos estilos. Lanzar un error (`GraphQLError`, lo que hacemos: aparece en la lista `errors`), o devolver los errores de validación **como datos** dentro de la respuesta (por ejemplo, un campo `errores: [String]`), que el cliente muestra al usuario como mensajes de formulario.
- **Tipos de entrada** (`input`): cuando una mutation recibe muchos argumentos, se agrupan en un tipo de entrada, como un serializer de DRF.

Ahora sí, creen el archivo que faltaba, `taskflow/config/schema.py` (el que referenciamos en la Parte 2.1):

```
from core.schema import schema  # noqa: F401
```

Este archivo solo "reexporta" el esquema de la app `core`, para que la configuración `GRAPHENE["SCHEMA"]` lo encuentre. El comentario `# noqa: F401` le avisa a los analizadores de código (como el de VS Code) que el import "sin usar" es intencional.

2.5 Exponer el endpoint

Graphene trae una vista lista, `GraphQLView`, pero es una vista de **Django común**, no de DRF: no lee el header `Authorization: Bearer ...` por su cuenta, así que con ella `info.context.user` sería siempre anónimo. La extendemos para que use la misma autenticación JWT de la Clase 6. Creen `taskflow/core/graphql_views.py`:

```
from graphene_django.views import GraphQLView
from rest_framework.exceptions import AuthenticationFailed
from rest_framework_simplejwt.authentication import JWTAuthentication


class JWTGraphQLView(GraphQLView):
    """
    GraphQLView es una vista de Django común, no de DRF: no lee el header
```

```
    Authorization por su cuenta. Acá lo hacemos a mano con la misma clase
    JWTAuthentication que usa DRF (Clase 6).
    """

    def dispatch(self, request, *args, **kwargs):
        try:
            result = JWTAuthentication().authenticate(request)
        except AuthenticationFailed:
            result = None
        if result is not None:
            request.user, _ = result
        return super().dispatch(request, *args, **kwargs)
```

- `dispatch` es el método que Django ejecuta al principio de toda vista basada en clase, antes de derivar a `get` o `post`. Lo sobreescribimos para autenticar primero y después dejar que siga el comportamiento normal (`super().dispatch(...)`).
- `JWTAuthentication().authenticate(request)` es exactamente lo que hace DRF en cada vista de la API: busca el header `Authorization`, valida el token y devuelve una tupla `(usuario, token)`, o `None` si no hay header.
- Si el token es inválido o está vencido, lanza `AuthenticationFailed`. Lo atrapamos y seguimos como anónimo, así los resolvers devuelven listas vacías.
- `request.user, _ = result` "desarma" la tupla: guarda el usuario en `request.user` y descarta el token (el `_` es la convención para "no me interesa este valor").

En `taskflow/config/urls.py`, agreguen los imports y la ruta:

```
from django.views.decorators.csrf import csrf_exempt
from core.graphql_views import JWTGraphQLView

urlpatterns = [
    # ... todo lo que ya tenían
    path("graphql/", csrf_exempt(JWTGraphQLView.as_view(graphiql=True))),
]
```

- `csrf_exempt` desactiva, para este endpoint puntual, la protección CSRF de Django. Esa protección defiende la autenticación por **cookies** de sesión; acá autenticamos con JWT en el header `Authorization`, que un sitio malicioso no puede enviar en nombre del usuario.
- `graphiql=True` habilita **GraphiQL**, una interfaz interactiva en el navegador para escribir y probar queries y mutations sin herramientas externas. Se accede entrando directamente a la URL del endpoint desde el navegador.

Parte 3 — Autenticación en GraphQL y prueba integral (55 min)

3.1 Cómo llega el usuario hasta el resolver

El recorrido completo de una petición:

1. El cliente manda `POST /graphql/` con el header `Authorization: Bearer <token>` y la query en el cuerpo.
2. `JWTGraphQLView.dispatch` valida el token y deja el usuario en `request.user`.
3. Graphene ejecuta los resolvers pasando ese mismo `request` como `info.context`.
4. Cada resolver lee `info.context.user` y aplica las reglas de membresía.

Resultado: GraphQL usa **la misma** autenticación (Clase 6), **los mismos** permisos (Clase 7), **las mismas** validaciones (Clases 4 y 8) y **las mismas** notificaciones (Clase 10) que la API REST. Solo cambia la forma de pedir los datos.

3.2 Probar desde GraphiQL

Abran `http://127.0.0.1:8000/graphql/` en el navegador y ejecuten primero, **sin** autenticación:

```
query {
  allProjects {
    id
    name
```

```
  }
}
```

Deberían ver `"allProjects": []`, por el `Project.objects.none()` del resolver.

Ahora obtengan un `access` token (Clase 6) de un usuario admin de algún proyecto. En GraphiQL, abajo del editor, abran la pestaña **Headers** y peguen (reemplazando el token):

```
{"Authorization": "Bearer <PEGAR_ACCESS_TOKEN_AQUI>"}
```

Repitan la consulta pidiendo también las tareas anidadas:

```
query {
  allProjects {
    id
    name
    tasks {
      title
      status
    }
  }
}
```

Y prueben una mutation (usen el `id` de un proyecto donde el usuario sea admin):

```
mutation {
  createTask(projectId: 1, title: "Probar GraphQL", priority: "alta") {
    ok
    task {
      id
      title
    }
  }
}
```

Si tienen abierto el `escuchar.py` de la Clase 10 sobre ese proyecto, van a ver llegar la notificación `task_created`: la tarea creada por GraphQL dispara el mismo evento que una creada por REST.

Prueben también los casos de error y observen la lista `errors` de la respuesta:

- `priority: "urgente"` → `Datos inválidos: {'priority': ['"urgente" is not a valid choice.']}`.
- Con el token de un usuario que es solo `miembro` del proyecto → `Solo un admin del proyecto puede crear tareas en él.`
- `task(id: ...)` con el `id` de una tarea de un proyecto ajeno → `"task": null`.

Si prefieren Postman o Insomnia: `POST` a `http://127.0.0.1:8000/graphql/`, header `Authorization: Bearer <token>`, y el cuerpo en formato JSON `{"query": "..."}`, o usando el modo GraphQL que traen esas herramientas.

> 💡 **Nota del docente**
> Noten que con **una sola petición** trajimos proyectos y, anidadas, sus tareas: algo que en REST resolvimos con el serializer anidado de la Clase 3 (`ProjectSerializer` incluyendo `TaskSerializer`). Pero acá quien decide **qué** anidar y hasta qué profundidad es el **cliente**, campo por campo, no el servidor de antemano.

3.3 Concepto: seguridad propia de GraphQL

Que el cliente arme la consulta trae riesgos que en REST no existen:

- **Consultas demasiado profundas o costosas:** como `ProjectType` tiene `tasks` y `TaskType` tiene `project`, alguien puede pedir `allProjects { tasks { project { tasks { project { ... } } } } }` a cualquier profundidad, y hacer trabajar al servidor sin límite (una denegación de servicio con una sola petición).

Las defensas son **limitar la profundidad** y **calcular un costo** por consulta antes de ejecutarla, y rechazar las que lo superen.

- **Introspección en producción:** la misma función que usa GraphiQL para autocompletar le muestra a un atacante todo el esquema. Muchas APIs la desactivan en producción.
- **Autorización en cada resolver:** una sola URL no se puede proteger "por ruta". Cada punto de entrada tiene que aplicar sus propias reglas: por eso usamos `tasks_visible_for` y `MembershipService` en todos los resolvers.
- **Límites de peticiones:** los de la Clase 8 se aplican a las vistas de DRF; nuestra `JWTGraphQLView` es una vista de Django común y no los tiene. Es un buen ejercicio agregárselos.

3.4 Concepto: integración de fuentes y federación de datos

Un resolver puede obtener sus datos de **cualquier lado**: la base de datos, otra API REST, un microservicio. Eso convierte a GraphQL en una buena **capa de unificación**: por ejemplo, un campo `resumenIa` en `ProjectType` cuyo resolver llame a `AIService.summarize_project` (clase de IA) haría que el cliente reciba, en una sola consulta, datos de PostgreSQL y de un modelo de IA, sin saber de dónde viene cada cosa. Es lo que el programa llama "integración de GraphQL con fuentes de datos existentes".

Cuando una organización tiene **varios** servicios, cada uno con su propia API, hay dos formas de ofrecerle al cliente un único esquema:

Enfoque	Cómo funciona
Unión de esquemas (*schema stitching*)	Un servidor central toma los esquemas de los otros servicios y los combina; él decide cómo se relacionan
Federación (*Apollo Federation* y similares)	Cada servicio publica su parte del esquema (un **subgrafo**) y declara qué entidades comparte; un **gateway** (o *router*) arma el esquema completo y reparte cada consulta entre los servicios



Con federación, cada equipo mantiene su servicio de forma independiente, y el cliente ve una sola API. Es la propuesta de actividad conjunta con la comisión de los jueves para la clase de integración de APIs.

Parte 4 — Guardar el avance en Git y GitHub (20 min)

```
git status
git add .
git commit -m "Agrega capa GraphQL con Graphene-Django sobre los modelos, permisos y validaciones
existentes"
git push
```

Parte 5 — Cierre y tarea

5.1 Resumen

- Diferencias prácticas entre REST y GraphQL (over-fetching / under-fetching, una URL vs. muchas, errores en el JSON vs. códigos HTTP).
- Types, Queries y Mutations con Graphene-Django sobre los mismos modelos desde la Clase 3.
- Reutilización de la autenticación JWT (vista `JWTGraphQLView`), de los permisos (`MembershipService`), de las validaciones (`TaskSerializer`) y de las notificaciones (`NotificationService`), sin duplicar lógica.

5.2 Tarea para la casa

1. Agreguen una mutation `UpdateTaskStatus` (siguiendo el mismo patrón que `CreateTask`) que reciba `id` y `status`, actualice la tarea correspondiente y valide que el usuario autenticado sea **admin** de su proyecto. Pistas: busquen la tarea con `tasks_visible_for(user)`; para validar, usen `TaskSerializer(tarea, data={"status": status}, partial=True)`, que aplica las reglas de la Clase 4 igual que un `PATCH`.
2. Comparen, con un ejemplo propio y por escrito, un caso donde usarían REST y uno donde usarían GraphQL en un proyecto real, justificando la elección con los conceptos de esta clase.
3. Guarden el avance repitiendo el flujo de la Parte 4.

5.3 Próxima clase — cierre de la materia

Clase 12: **testing automatizado, CI/CD y despliegue** en una plataforma cloud: la última pieza para que TaskFlow deje de vivir solo en `localhost`.

Errores frecuentes

- **`allProjects` siempre vacío aunque mando el token**: la ruta usa `GraphQLView` en vez de `JWTGraphQLView`, o el header está mal escrito en la pestaña Headers (tiene que ser JSON válido, con comillas dobles).
- **`Cannot query field "all_projects" on type "Query"`**: en GraphQL se escribe `allProjects` (camelCase).
- **`ImportError: Could not import 'config.schema.schema'`**: falta el archivo `config/schema.py`, o tiene un error de sintaxis.
- **`status` sale en mayúsculas (`PENDIENTE`)**: falta `convert_choices_to_enum = False` en `TaskType`.

Preguntas de repaso

1. ¿Qué es el SDL? Lean el esquema de TaskFlow y expliquen qué significa `[TaskType!]!`.
2. ¿Cuáles son los tres tipos de operación de GraphQL? ¿Cuál se relaciona con la Unidad 4?
3. ¿Por qué una respuesta GraphQL puede traer `data` y `errors` a la vez? ¿Qué código HTTP suele usar?
4. Comparen REST y GraphQL en caché, versionado y manejo de errores.
5. ¿Cómo se ejecuta una consulta como árbol? ¿Qué recibe cada resolver en `root` y en `info.context`?
6. ¿Qué es el problema N+1 en GraphQL? ¿Cuántas consultas SQL medimos con 6 proyectos, y cuántas con `prefetch_related`?
7. ¿Qué convenciones conviene seguir al diseñar una mutation?
8. ¿Qué riesgos de seguridad son propios de GraphQL y cómo se mitigan?
9. ¿Cómo puede un resolver integrar datos de otro servicio? Den un ejemplo con TaskFlow.
10. ¿Qué diferencia hay entre unir esquemas y federar? ¿Qué hace un gateway?

Para profundizar

- Sitio oficial de GraphQL, con el curso "Learn": `graphql.org/learn/`
- Documentación de Graphene-Django: `docs.graphene-python.org/projects/django/`
- Especificación de GraphQL: `spec.graphql.org`
- Documentación de Apollo Federation: `apollographql.com/docs/federation/`
- OWASP, *GraphQL Cheat Sheet*: `cheatsheetseries.owasp.org`

Diccionario de términos de esta clase

- **camelCase:** convención de nombres que junta palabras con mayúscula inicial desde la segunda (`allProjects`); es la usada en GraphQL.
- **Contexto de ejecución:** información compartida por todos los resolvers de una consulta; en Graphene-Django, el `request` con el usuario.

- **DataLoader:** patrón que agrupa los pedidos de datos de un mismo nivel en una sola consulta, para evitar el problema N+1.
- **Federación:** técnica para combinar las APIs GraphQL de varios servicios en un único esquema, a través de un gateway.
- **Gateway (router):** servidor que recibe las consultas de los clientes y las reparte entre los servicios que forman el esquema.
- **GraphiQL:** interfaz web interactiva, incluida en Graphene-Django, para probar queries y mutations desde el navegador.
- **Mutation:** en GraphQL, una operación que modifica datos (equivalente conceptual a POST/PUT/DELETE en REST).
- **Over-fetching:** traer más datos de los que el cliente realmente necesita.
- **Query (en GraphQL):** una operación de lectura de datos.
- **Resolver:** función que sabe cómo obtener el valor concreto de un campo de GraphQL.
- **Resultado parcial:** respuesta que incluye los datos que se pudieron obtener y la lista de errores de los que fallaron.
- **Schema (esquema):** definición completa de qué se puede consultar y modificar en una API GraphQL.
- **SDL (Schema Definition Language):** lenguaje para escribir esquemas GraphQL.
- **Subscription:** operación de GraphQL para recibir datos en tiempo real.
- **Tipo escalar:** tipo de dato simple de GraphQL (`Int`, `String`, `Boolean`, `ID`, `Float` y otros definidos).
- **Under-fetching:** no traer suficientes datos en una sola petición, obligando a hacer peticiones adicionales.